Autonomous-QA Porting Playbook — Lava → Boba-Base

Deep research of every solution/approach developed in the Lava autonomous-QA effort, structured for porting into **Boba-Base** (`git@github.com:milos85vasic/Boba-Base.git`). Each section: **Problem → Root cause → Solution → Key files/code → Port steps**. Ordered by foundational dependency.

The end goal these solve: drive a **clean Android install → onboard → search → open details → obtain a download (magnet/torrent/HTTP)** flow on a *containerized* emulator, against a real backend, reaching real upstream providers — recorded, with anti-bluff verdicts — for many provider/backend/query combinations.

---

0. The single most important architectural insight (read first)

There are **two distinct egress paths**, and which one a provider uses determines where you must inject reachability:

1. **Backend-routed (RemoteTrackerDescriptor / API-backed)** — the app calls *your backend* (loopback via `adb reverse`); the **backend** makes the upstream call. To reach blocked upstreams, proxy the **backend** (§3).
2. **On-device-direct (bundled descriptor)** — the app's *bundled client* (e.g. `RuTrackerHttp`) hits the tracker **directly from the device**. The backend proxy is irrelevant here; the **emulator's own egress** must reach the upstream (§2 + §4).

The onboarding picker **falls back to bundled descriptors when the API `/v1/providers` catalogue fetch fails**, silently switching the whole flow to on-device-direct. So a "backend" test can quietly become a direct test. **Diagnose which path is active before debugging reachability.** (Forensic: logcat `RuTrackerHttp: REQUEST https://rutracker.org/...` proves on-device-direct.)

---

1. Containerized Android emulator (podman/docker, KVM)

Problem: need a reproducible emulator that runs *inside* a container (not host-direct), with `adb` reachable from the host/test runner.

Solution (the working recipe):

- Image bundles Android SDK + AVD + a baked adb keypair + a **socat bridge** (container `0.0.0.0:5575` → `127.0.0.1:5555` adbd) so adb is reachable without host networking.
- Boot: `podman run -d --rm --userns=keep-id --device /dev/kvm -p 127.0.0.1:<hostADB>:5575 -p 127.0.0.1:<hostConsole>:5554 -e ANDROID_AVD_NAME=default -e ANDROID_COLD_BOOT=true <image>`
- Host adb: copy the baked key out (`podman cp <c>:/home/emulator/.android/adbkey`), `export ADB_VENDOR_KEYS=…`, `adb connect 127.0.0.1:<hostADB>`.
- Poll readiness: `getprop sys.boot_completed == 1` AND `pm path android` succeeds.

Key files: `scripts/autonomous-qa/lib-emulator.sh` (`emu_boot/emu_authorize_adb/emu_connect/emu_wait_boot/emu_cleanup_orphans/emu_teardown`) — thin glue replicating `submodules/containers/pkg/emulator/containerized.go`. Image: `ghcr.io/vasic-digital/lava-android-emulator:api34-x86_64` (Containerfile in the Containers submodule).

CRITICAL — podman version: podman 5.7.1 (pasta) boots the emulator with **working guest networking natively**; **podman 4.9.3 (slirp4netns)** boots but the **guest has no network** (see §2). Prefer ≥5.x on the gate host. The container's network backend does NOT fix the *guest* route (proven by testing host/caps+tun/pasta — all identical guest result); the divergence is between the podman versions' interaction with the emulator, addressed by §2's guest-side fix regardless.

Port steps: reuse the Containers submodule's emulator package + image; copy `lib-emulator.sh` verbatim (it's generic). On TMPDIR-constrained hosts set `TMPDIR=$HOME/.podman-build-tmp` for image builds (32GB `/tmp` tmpfs filled during build).

---

2. Containerized-emulator guest networking fix (the `ndc` dance)

Problem: on some hosts the emulator boots but the guest has **no network** — `ip route` empty, `ping 10.0.2.2` "Network is unreachable", onboarding stalls.

Root cause: Android never registers `eth0` with its framework at boot (the boot-time interface-add fails while eth0 is still DOWN). Android uses **per-network mark-based routing** (netd) — so a raw `ip route` on the main table is **ignored** by apps. The NIC is fine; the *framework* doesn't know the network exists. (`eth0` shows `state DOWN`; forcing it up gives carrier+IP but apps still see "unreachable" because netd routing isn't configured.)

Solution (post-boot, as root via `su 0`), retry-until-gateway-pings:

```
ip link set eth0 up
ip addr add 10.0.2.15/24 dev eth0
ndc network interface add 100 eth0      # net 100 = Android's
default physical net
ndc network default set 100
ndc network route add 100 eth0 0.0.0.0/0 10.0.2.2   # run AFTER
eth0 up
setprop net.dns1 10.0.2.3
```

Verify `ping -c2 10.0.2.2` succeeds. Constants are **platform-fixed** Android-emulator slirp values: `10.0.2.2`=host/gateway, `10.0.2.3`=DNS, `10.0.2.15`=guest (the emulator's `127.0.0.1` equivalent). Wrap in an 8-attempt retry loop (boot-timing race) and **self-gate**: if `ping 10.0.2.2` already works (healthy podman 5.x guest), no-op.

Key code: `scripts/autonomous-qa/lib-emulator.sh` → `emu_fix_network()`. Note: `ping 8.8.8.8` failing is EXPECTED (slirp doesn't NAT ICMP-to-internet) — TCP/UDP work; don't use ICMP-to-internet as a health signal.

Port steps: copy `emu_fix_network`; call it once after `emu_wait_boot`. It's a no-op where unneeded, so it's safe everywhere.

---

3. Backend configurable outbound proxy (for backend-routed providers)

Problem: the backend's upstream calls to providers go out the host's IP; if that IP is blocked you need to route through a different egress.

Solution: a single env knob `LAVA_API_UPSTREAM_PROXY` (`socks5://` | `http://` | `https://`), falling back to standard `HTTP_PROXY`/`HTTPS_PROXY`/`ALL_PROXY`/`NO_PROXY` when unset. A shared transport factory sets `http.Transport.Proxy` on **every** provider client; **loopback bypass** keeps local sidecars (Jackett at 127.0.0.1) direct. Go's `net/http`

dials `socks5://` natively (no `x/net/proxy` dep) and does **remote DNS** for socks5 (critical — the host's DNS is often part of the block).

Key files: `lava-api-go/internal/httpx/proxy.go` (`ProxyFunc`/`Configure`/`Proxy`/`NewTransport`), wired in `cmd/lava-api-go/main.go` (`httpx.Configure(cfg.UpstreamProxy)` BEFORE provider construction) + each `internal/<provider>/client.go` (`Proxy: httpx.Proxy` / `Transport: httpx.NewTransport()`); config in `internal/config/config.go`; documented in `.env.example`. Tested with a live local proxy + real provider clients + falsifiability (mutate `Proxy→nil` → test fails).

Deployment wiring gotcha: the deploy script must **forward the env into the container** (allow-list). See `deployment/thinker/thinker-up.sh` — `LAVA_API_UPSTREAM_PROXY` (+ proxy fallbacks) added to the `for var in … -e` loop. With `--network host`, the container reaches the host's `127.0.0.1:<proxyport>`.

Verify env reached the container: the image is **distroless** (no shell/`printenv`) → `podman exec … printenv` is a **false-negative**. Use `podman inspect <c> --format '{{range .Config.Env}}{{println .}}{{end}}'`.

Port steps: copy the `httpx` proxy factory pattern into Boba-Base's backend HTTP layer; add the config knob + `.env.example` entry + the deploy-script env-forward.

---

4. Provider/upstream egress via a VPN-connected host (the nezha pattern)

Problem: datacenter/cloud host IPs are **network-level blocked** for the Russian trackers — DNS-resolution failure + TLS-MITM (self-signed cert) + connection-refused. This is **ISP/DPI blocking, NOT a Cloudflare challenge** → FlareSolverr/Jackett **cannot** bypass it. (Diagnose by: `curl https://api.ipify.org` for the host IP; `curl -o /dev/null -w %{http_code} https://rutracker.org/…` direct → fails; same via a VPN-host proxy → 200.)

Solution: route through a **VPN-connected host** (`nezha.local`, VPN egress IP reaches rutracker/nnmclub/archive.org = 200). Two sub-approaches by egress path (§0):

- **Backend-routed:** a SOCKS5 tunnel + `LAVA_API_UPSTREAM_PROXY` (§3). Tunnel from the backend host to the VPN host: `ssh -D 127.0.0.1:1080 -N <vpnhost>` (use `--socks5-hostname` for remote DNS). The tunnel exits via the VPN host's network (its VPN). Set `LAVA_API_UPSTREAM_PROXY=socks5://127.0.0.1:1080`.
- **On-device-direct:** the tunnel/backend-proxy does NOT help (the emulator egresses, not the backend). **Run the whole**

matrix ON the VPN host — there the emulator's slirp NAT exits through the VPN host's network → VPN → upstream. (Requires podman+KVM+Android-SDK+the emulator image on the VPN host.)

Key facts to port: the diagnosis script (host-IP + per-tracker HTTP code, direct vs via-proxy); the decision rule (which egress path → which fix). Constants/coords live in `.env` (gitignored) — never hardcode.

---

5. Durable remote execution (the operational unblock)

Problem: long (~20 min) remote runs kept dying — tmux sessions vanished, `nohup`/`setsid`/`disown` processes died, background SSH pollers were killed.

Root cause: the remote host's `systemd-logind` reaps **all** user processes when the SSH session ends (default `KillUserProcesses` behavior on that host) — so anything tied to the session dies, even detached.

Solution: enable lingering + run as a systemd user service:

```
export XDG_RUNTIME_DIR=/run/user/$(id -u)
loginctl enable-linger                       # processes persist
past session end
systemd-run --user --unit=<name> --collect bash /path/to/runner.sh
systemctl --user is-active <name>            # poll; result
persists on disk
```

This was the ONLY mechanism that survived (tmux/nohup all failed). Poll by checking `is-active` + the on-disk evidence (don't rely on a long-lived SSH poller — those get killed by the local harness too; keep pollers short, or check on the next turn).

Port steps: any long remote job (matrix, build) → wrap in a self-contained runner script, scp it over, launch via `systemd-run --user` with linger. Write a `MATRIX_COMPLETE` sentinel for completion detection. **Gotcha:** if the runner pipes a long command through `tail -N`, output buffers until exit — log progress separately (per-iteration `verdict.json`) instead of relying on the piped log.

---

6. The autonomous-QA harness (orchestration)

Structure (all in `scripts/autonomous-qa/`):

- `lib-subsets.sh` — emits the provider-mix matrix (all 2^n-1 non-empty subsets of the backed-provider set, as `csv|slug|statehash`).
- `lib-emulator.sh` — containerized emulator lifecycle (§1) + `emu_fix_network` (§2).
- `lib-backend.sh` — `backend_up_<goapi|apiapp>` / `backend_target_*` / `backend_down_*` with a **mutual-exclusion lock** ("never run both backends at once"). The on-device backend (`apiapp`) embeds the Go server (`:core:apiengine`) and is started with `am start … --ez <START_API_extra> true` + `pm grant POST_NOTIFICATIONS` (a plain LAUNCHER launch leaves it stopped); readiness = real `/health` poll over TLS via `adb forward`, not `podman ps` "Up" (§6.B: "Up" ≠ healthy).
- `run-iteration.sh` — **fresh install** (uninstall+install -r for clean onboarding state) → start recording (logcat + chunked `screenrecord`) → `./gradlew :app:connectedDebugAndroidTest -Pandroid.testInstrumentationRunnerArguments.<key>=<val>` → parse JUnit → marker-based verdict.
- `run-matrix.sh` — backend up → boot ONE kept-alive emulator → `emu_fix_network` → loop subsets×queries → teardown. `--external-backend` skips backend lifecycle (when the backend is managed separately).
- `lib-remote.sh` — `remote_sync_repo` (rsync incl. gitignored `.env`, exclude build/.git/raw/recordings) + `remote_run "<cmd>"` + `remote_fetch`. Targets a host from `.env` (`LAVA_API_GO_REMOTE_HOST`); override per host.
- `vision-analyze.sh` / `aggregate-evidence.sh` / `lib-jackett.sh`.

Anti-bluff verdict (the load-bearing bit): the test logs `C70-RESULT … DOWNLOAD-OK` **only** after the real on-screen download/magnet affordance is confirmed. `run-iteration.sh` greps logcat+JUnit and applies a decision table: PASS only if the marker is present AND the only failure is the known teardown crash (LVA-008) AND no other failure signal. Verdicts: PASS / SKIP (honest `assumeTrue` skip when env/provider unreachable) / FAIL. **Known bug to fix on port:** the summary counter regex `grep -oE '[A-Z]+$'` never matches the trailing-`"` JSON token → mislabels everything FAIL; use `grep -oE '"verdict": *"[A-Z]+"' | grep -oE '[A-Z]+' | tail -1`. Per-iteration `verdict.json` is authoritative.

Port steps: copy the whole `scripts/autonomous-qa/` tree; adjust `lib-subsets` provider set, the instrumentation-arg names, and the marker tag to Boba-Base's test.

---

7. The parameterized Compose UI Challenge (the device test)

Pattern: `createAndroidComposeRule<MainActivity>` + a single `@Test` that reads instrumentation args (`qa_backend`/`qa_providers`/`qa_query`/`qa_api_url`) and drives the **real production path**: onboarding wizard → provider pick → configure/login → search → results → topic → download. (`app/src/androidTest/kotlin/lava/app/challenges/Challenge70AutonomousQaProviderMatrixTest.kt`.)

Hard-won fixes to port:

- **Off-main navigation crash** (`IllegalStateException: setCurrentState must be called on the main thread`): the test harness's `FrameDeferringContinuationInterceptor` resumes `collectSideEffect`/`LaunchedEffect` coroutines **off the main thread**, so `navController.navigate/navigateUp/popBackStack` crash. Fix in the nav layer: marshal every nav call to the main looper. `navigate()` (returns Unit) → `runOnMainThread { … }` (fire-and-forget Handler post); `popBackStack()` (returns Boolean) → `runOnMainThreadResult { … }` (CountDownLatch blocking marshal, propagates result+exceptions). No-op in production (already on main). See `core/navigation/.../NavigationController.kt`.
- **Provider deselection**: the picker renders **bundled descriptor** displayNames (e.g. `RuTracker.org`, `RuTor.info`, `IPTorrents`), NOT the API/humanized names — enumerate the *real* rendered names (grep `core/tracker/*/src/main/**/*Descriptor.kt`) and deselect all non-requested ones, else a credentialed provider's Configure page blocks the wizard.
- **All-affordance download handling**: the "Torrent" button is the download affordance for **all** non-magnet providers *including HTTP-file ones* (it routes through `onTorrentFileClick → resolveProviderDownloadKind → downloadHttpFile|downloadTorrentFile`, both → the same `DownloadDialog`). Don't invent a "file-row" matcher (doesn't exist). Confirm download via `DownloadDialog` text "Download completed"/"Downloading file…" (NOT "Download in progress" — that string doesn't exist). Magnet → `MagnetDialog` "Open".
- **Transient-search retry**: on the "Search failed / problem reaching the trackers" state, tap "Retry" up to 3× before failing loudly (handles upstream flakiness).
- **Honest skips**: wrap onboarding in `try { … } catch (ComposeTimeoutException) { assumeTrue(false, "…honest §6.J skip…") }` — an unreachable env/provider is a SKIP, never a green.

---

8. Jackett + FlareSolverr sidecar (Cloudflare-protected trackers)

Use when: the block is a **Cloudflare challenge** (not an ISP/DPI block — §4). FlareSolverr solves the challenge; Jackett aggregates indexers and exposes Torznab.

Backend integration: `LAVA_API_JACKETT_ENABLED/URL/APIKEY/DEFAULT_INDEXER`. Sidecar via `scripts/autonomous-qa/lib-jackett.sh` + `tools/lava-containers/docker-compose.jackett.yml` (Jackett 9117, FlareSolverr 8191).

§6.J gap to port-fix: a **fresh Jackett has zero indexers** (must be added via its API), and its **management API needs a dashboard session cookie** — the apikey only authorizes the Torznab `/results+/caps` feeds (management GET/POST → 302 `/UI/Login`). The backend's jackett client must do the **empty-password cookie login** (POST `/UI/Dashboard`, cookie jar) for `ListIndexers`/config; apikey stays for Torznab. And the **test fake must enforce the same 302-without-cookie** behavior (behavioral-equivalence / Third Law) or the gap stays hidden. See `lava-api-go/internal/jackett/`.

---

9. Diagnostic techniques (reusable)

- **`C70-TREE` semantics dump**: on any failure, log `composeRule.onRoot(useUnmergedTree=true).printToString(maxDepth=100)` → recover the exact on-screen state (e.g. an `[Error] / Something went wrong / Retry` topic screen) from the recording even after the activity tears down.
- **Frame extraction from screenrecord**: `ffmpeg -ss <t> -i rec.mp4 -frames:v 1 frame.png`; small PNGs = simple/blank screens (loading/error), large = populated — a cheap progress signal.
- **distroless env check**: `podman inspect` (not `exec printenv`).
- **buffered `tail` trap**: a command piped through `tail -N` shows nothing until it exits — read per-iteration artifacts for live progress.

---

10. Honest status of the *remaining* (per-provider, post-egress) blockers

After egress is solved, the search→download still depends on per-provider realities — port these expectations:

- **CAPTCHA_LOGIN providers (rutracker)**: cannot be auto-onboarded (captcha). Honest SKIP; needs a manual-creds/session-cookie path or a captcha service.
- **archiveorg topic-detail bug**: search works, but opening a result shows "Something went wrong" consistently (independent of egress) — a real topic-fetch/parse bug to fix in the provider client.
- **FORM_LOGIN providers (nnmclub/kinozal)**: should auto-onboard once egress works; trace the login step if onboarding times out.
- Anonymous, well-behaved providers (e.g. a working Gutenberg/Archive client) are the cleanest first-green target.

Recommended porting order into Boba-Base

1. §1 containerized emulator + §2 guest-net fix (foundation).
2. §5 durable execution (so long runs survive) + §6 harness skeleton.
3. §7 the parameterized Challenge + the 4 device-test fixes.
4. §0/§4 egress decision + §3 backend proxy / VPN-host run.
5. §8 Jackett (only if Cloudflare-blocked) + §9 diagnostics + §6 verdict-counter fix.

> To tailor this to Boba-Base's actual structure (module names, provider clients, deploy scripts), add the repo to the session and I'll map each section to concrete Boba-Base files + generate the diffs.